

Curso Programación en C microcontroladores PIC

Introducción al MPLAB C-18

Andrés Prieto-Moreno Torres

andres@ifara.com

- Compilador ANSI C para microcontroladores PIC18.
- Aprovecha las modificaciones arquitecturales de la familia de microcontroladores PIC18.
 - Diferente distribución memoria de datos.
 - Nuevas instrucciones de acceso a tablas en memoria de programa
 - Nuevos modos de direccionamiento.
- Admite ensamblador empotrado
- Integrado con el MPLAB-IDE
- Versión de DEMO gratuita

Tipo	Tamaño	Mínimo	Máximo
Char	8 bits	-128	127
Signed char	8 bits	-128	127
Unsigned char	8 bits	0	255
Int	16 bits	-32768	32767
Unsigned int	16 bits	0	65535
Short	16 bits	-32768	32767
Unsigned short	16 bits	0	65535
Short long	24 bits	-8.388.608	8.388.607
Unsigned short long	24 bits	0	16.777.215
Long	32 bits	-2.147.483.648	2.147.483.647
Unsigned long	32 bits	0	4.294.967.295

Tipo	Tamaño	Exponente Mínimo	Exponente Máximo
Float	32 bits	-126	128
Double	32 bits	-126	128

Los datos de más de un byte de longitud, se almacenan en memoria siguiendo el criterio **LITTLE ENDIAN**, es decir los bytes menos significativos ocupan las posiciones de memoria más bajas.

Ejemplo:

```
#pragma idata mi_dato=0x1000
Long valor = 0xAABBCCDD;
```

0x1000	0xDD
0x1001	0xCC
0x1001	0xBB
0x1003	0xAA

MPLAB C18 maneja los modificadores establecidos por la norma ANSI:

1. Auto:

- Las variables declaradas fuera de las funciones son globales y las declaradas en las funciones son locales. El ámbito local tiene prioridad sobre el global.
- Si no se inicializan el valor que toman es indefinido.
- Las globales se comportan como estáticas en el bucle C018i.c (*)

2. Extern:

- Indica que la variable declarada pertenece a otro módulo, por lo que no es necesario reservar memoria para ella.
- Dentro de un mismo fichero fuente sirve para acceder a una variable aunque todavía no se haya llegado a su declaración.
- Se auto-inicializan a cero.

3. Register:

- Indica que la variable declarada se debe guardar en uno de los registros del procesador.
- Si no es posible, se comporta como 'Auto'
- Se utiliza para optimizar el tiempo de ejecución de algunas funciones.

4. Static:

- Variables **locales** a una función que retienen su valor en llamadas sucesivas a dicha función.
- Hay que poner **static** delante de la declaración.
- Se inicializan a valor cero.
- Ahorra número de instrucciones para acceder a las variables.

4. Typedef:

- Asigna un nuevo nombre a un tipo de datos.

Y además añade ...

5. Overlay:

- Se aplica sobre variables locales.
- Las declara estáticas pero se inicializan cada vez que se llama a la función.
- Se permite que una misma posición de memoria pueda ser utilizada por diferentes variables. Podemos ahorrar espacio de memoria.
- El proceso es controlado por el *linker*.

1. **Const:** Contenido de la variable es fijo. ANSI C.
2. **Volatile:** El contenido de la variable puede cambiar. ANSI C.

MPLAB C-18 añade ...

3. **Far:**
 - **Datos:** La variable esta en un banco lejano, se necesitan instrucciones de cambio de banco y/o acceso indirecto.
 - **Código:** Podemos acceder a objetos por encima de los 64K.
4. **Near:**
 - **Datos:** La variable esta en el banco de Acceso. (acceso directo mediante 'a').
 - **Código:** El objeto esta en una zona menor de 64K.

5. **RAM:**
 - La variable se sitúa en en la memoria de datos.
6. **ROM:**
 - La variable esta en la memoria de programa.

Ejemplos:

- *Variable en memoria de datos : char data;*
- *Variable en código cercano : rom near char data;*
- *Variable en código lejano : rom far char data;*

Por defecto FAR y RAM

```

1. // Definimos un tipo nuevo
2. typedef unsigned char BYTE;

3. BYTE var_global = 0x0; // la primera zona de memoria GPR0 ( 0x80 )

4. // programa principal
5. void main(void) {
6.     extern int var_externa; // declarada externa
7.     BYTE var_auto = 0x40; // variable local auto (PILA -> 0x500 )
8.     static BYTE var_estatica = 0x50; // variable local estática ( 0x80 )

9.     // programa principal
10.    for (;;) {
11.        var_auto++ ; // 0x500 ( PILA -> 0x500 )
12.        var_estatica++ ; // 0x8x / con NEAR 0x0x
13.        var_global++ ; // 0x8x / con NEAR 0x0x
14.        var_externa++ ; // 0x8x / con NEAR 0x0x
15.        if (var_estatica==0x60) { break; }
16.    }
17. }
18. // Ejemplo de declaración de una variable extern
19. unsigned int var_externa; // la primera zona de memoria GPR0 ( 0x80 )
    
```

- Repetitivas:
 - While
 - do-while
 - For
- Selectivas:
 - If
 - If-else
 - Switch
- Bifurcación de control:
 - Break, continue, goto
 - Return
 - Exit: No válido en MPLAB-C18

Sentencia **WHILE**:

```
while ( expresión ) {
    Instrucción 1
    Instrucción 2
    ...
    Instrucción n
}
```

- Las instrucciones se ejecutan repetidamente hasta que el valor de la expresión sea falso, es decir cero.
- La expresión se evalúa al principio

Sentencia **DO**:

```
do {
    Instrucción 1
    Instrucción 2
    ...
    Instrucción n
} while ( expresión );
```

- Las instrucciones se ejecutan repetidamente hasta que el valor de la expresión sea falso, es decir cero.
- La expresión se evalúa al final

Sentencia **FOR**:

```
for ( exp1; exp 2; exp 3) {
    Instrucción 1
    Instrucción 2
    ...
    Instrucción n
}
```

- Las instrucciones se ejecutan repetidamente hasta que el valor de la **exp2** sea falso, es decir cero.
- **exp1** sirve para inicializar el contador del bucle.
- **exp3** controla el contador del bucle. Se evalúa al final del mismo.

Ejemplo:

```
suma = 0;
for ( i=0; i<=100; i++) {
    suma=suma + i ;
}
```

Ejemplo de bucle infinito:

```
for (;;) {
    instrucciones;
}
```

Sentencia **if**

```
If ( expresión ) {
    Instrucción 1
    Instrucción 2
    ...
    Instrucción n
}
```

- La expresión se evalúa y si es verdadera (distinta de cero) se ejecutan las instrucciones.

Sentencia **if-else:**

```
If ( expresión ) {
    Instrucciones A
} else {
    Instrucciones B
}
```

- La expresión se evalúa, si es verdadera se ejecutan las instrucciones A, en caso contrario las Instrucciones B.

Sentencia **switch**

```
switch ( expresión ) {
    case const1: Instrucción 1;
        break;
    case const2: Instrucción 2;
        break;
    ...
    default: Instrucción n
        break;
}
```

- Permite realizar selecciones múltiples.

```
switch ( var ) {
    case 1: valor=0x55;
        break;
    case 2: valor=0x34;
        break;
    default: valor = 0x0;
        break;
}
```

Sentencia **break**

- interrumpe la ejecución de un bucle **while**, **do-while** o **for**.

Ejemplo: ¿ cómo salir de un bucle infinito **for** ?

```
For (;;) {  
    if ( a==0) {  
        break;  
    }  
    a=PORTA;  
}
```

Sentencia **continue**

- se utiliza en los bucles para pasar a la siguiente repetición.

```
For (;;) {  
    if ( a!=0) {  
        continue;  
    }  
    break;  
}
```

Sentencia **goto**

Transfiere incondicionalmente el control a la sentencia etiquetada por el identificador.

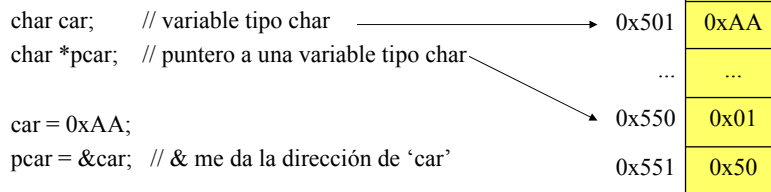
goto *identificador*;

identificador:
instrucciones;

!!! No es una buena práctica utilizarla !!!

Punteros a memoria I

- Un puntero es una variable que contiene la dirección a una zona de memoria donde reside un determinado tipo de dato.



¡ Los punteros a memoria de datos ocupan 16 bits !

- Otra forma de acceso a los datos:

```
char *pcar; // puntero a una variable tipo char
```

```
pcar = (char *) 0x501;
```

```
*pcar = 0xAA;
```

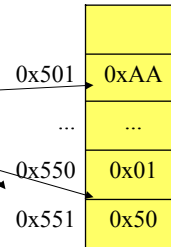
- Optimizando:

```
#define CAR (*(char *) 0x501)
```

```
CAR = 0xAA;
```

Nos hemos ahorrado las posiciones de memoria 0x550-0x551

Hay que tener cuidado, el compilador no chequea conflictos.



- En la memoria de programa se pueden almacenar variables, y por lo tanto se podrán definir punteros a ellas:

```
rom near unsigned char *pcar; // NEAR -> 16 bits
```

```
rom far unsigned int *pint; // FAR -> 24 bits
```

Un puntero de acceso a RAM no sirve para acceder a la ROM

- Operaciones matemáticas:
 - La norma ANSI establece que las operaciones matemáticas tienen que hacerse con precisión entera (INT) o superior. En este caso sería de 16 bits. El C18 permite operaciones entre CHAR con resultado un CHAR. Es decir 8 bits.
- Cadenas constantes:
 - Podemos almacenarlas en la memoria de programa.
 - Instrucciones especiales para moverlas entre la RAM y la ROM:

```
char *strcpy (auto char *s1, auto const char *s2)
char *strcpypgm2ram (auto char *s1, auto rom const char *s2)
rom char *strcpyram2pgm (auto rom char *s1, auto const char *s2)
rom char *strcpypgm2pgm (auto rom char *s1, auto const rom char *s2)
```

```
1. // Ejemplo de tratamiento de cadenas en RAM y ROM
2.
3. #include <string.h>
4. // almaceno mi cadena en ROM
5. const rom unsigned char micadena[]="hola mundo";
6.
7. // almaceno copia en RAM
8. char strbuffer[21];
9.
10. void main(void) {
11.     strcpy(strbuffer,micadena); // incorrecto
12.     strcpypgm2ram(strbuffer, micadena); // correcto
13.     for (;;)
14. }
```

Estructuras anónimas dentro de uniones



Se permite la declaración de estructuras anónimas dentro de las uniones. La utilidad principal es el acceso a bits.

```
union tpuerto {  
    unsigned char valor;  
    struct {  
        unsigned bit0:1;  
        unsigned bit1:1;  
        unsigned bit2:1;  
        unsigned bit3:1;  
        unsigned bit4:1;  
        unsigned bit5:1;  
        unsigned bit6:1;  
        unsigned bit7:1;  
    };  
    struct {  
        unsigned nibblel:4;  
        unsigned nibbleh:4;  
    };  
} byte;
```

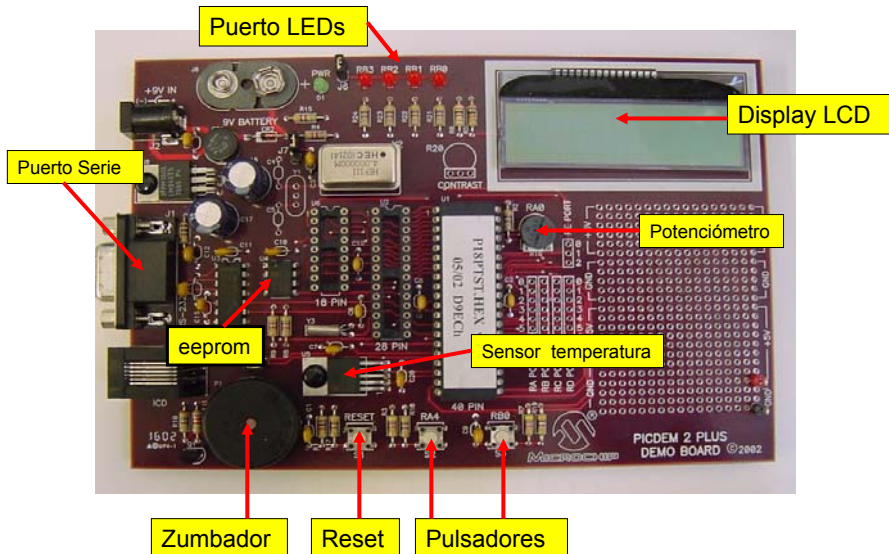
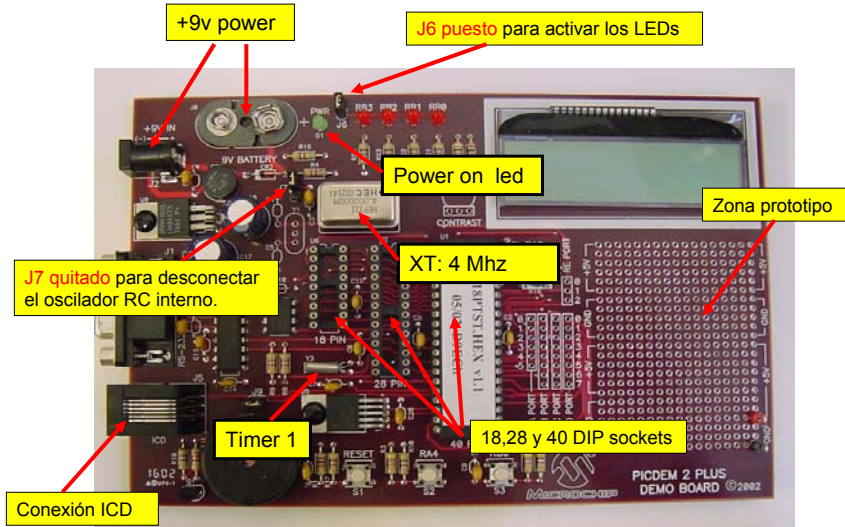
Ejemplo de uso:

```
// byte  
byte.valor=0xFF;  
  
// bits  
byte.bit0=0;  
byte.bit7=byte.bit0;  
  
// grupos de bits  
byte.nibblel=0x5;  
byte.nibbleh=0xa;
```



Curso Programación en C microcontroladores PIC

Placa entrenadora PICDEM 2



- Configuración de los Jumpers:
 - J6 puesto: Activa los LEDS
 - J7 quitado: Desactiva oscilador RC. Vamos a utilizar el Oscilador de 4Mhz.
 - J9 puesto: Activa el zumbador.
- PIC18F452 en su zócalo correspondiente. Función del número de patas.
 - Mantiene muy alta compatibilidad con PIC16F87X.
- Alimentación a +9v
 - El LED ver de Power On se debe de activar.
- Grabación en circuito
 - Usamos el conector telefónico para unir la PICDEM al ICD2.
 - Se ‘utilizan los pines’ RB6 y RB7

- Utilizamos el PIC18F452.
- LEDS: Conectados al Puerto B <3:0>
 - Periférico de salida.
 - El valor se lee o escribe accediendo al PORTB (0xF81)
 - TRISB (0xF93): Controla dirección de los bits del Puerto B.
 - 0 salida
 - 1 entrada
- SWITCHES: Conectados al Puerto A (RA4) y Puerto B (RB0)
 - Periférico de entrada
 - PORTB y TRISB controlan el acceso a RB0
 - PORTA (0xF80) y TRISA (0xF92) controlan acceso a RA4
 - 0 salida
 - 1 entrada

- Potenciómetro: Conectado al Pin RA0
 - Periférico de entrada.
 - Conectado entre GND y VDD (+5v)
 - Precisión de 10bits.
 - ADRESH (0xFC4): Guarda el byte alto del resultado
 - ADRESL (0xFC3): Guarda el byte bajo del resultado
 - ADCON0 (0xFC2): Registro de control del módulo A/D.
 - Bit 7-6 = 10 -> Frecuencia conversor Fosc/32
 - Bit 5-3 = 000 -> Seleccionamos canal 0 (RA0)
 - Bit 2 -> Status bit (0 conversión terminada / 1 empezar conversión en curso)
 - Bit 1 -> No implementado (cero)
 - Bit 0 -> Activación del conversor (0 AD OFF / 1 AD ON)
 - ADCON1 (0xFC1): Registro que controla las funciones de los pines del puerto.
 - Bit 7 -> Justificación (1 = Right / 0 = Left)
 - Bit 6 = 0 -> Frecuencia conversor Fosc/32
 - Bit 5-4 -> No implementados
 - Bit 3-0 = 1110 -> RA0 analógico y resto del puerto digital
 - ADIF / ADIE: Flag y máscara de la interrupción del conversor

- Zumbador: Conectado al Pin RC2
 - Periférico de salida
 - El valor se lee o escribe accediendo al PORTC (0xF82)
 - TRISC (0xF94): Controla dirección de los bits del Puerto C.
 - 0 salida
 - 1 entrada
- Display LCD: Conectado al Puerto A <3:1> y Puerto D <3:0>
 - Periférico de salida. Display de 2 líneas y 16 caracteres.
 - Puerto A (porta/trisa): Líneas de control.
 - Puerto D (portd 0xF83 / trisd 0xF95): Líneas de datos.
- Dispositivos I2C (eeprom y TC74)
 - Puerto C (PORTC / TRISC): bits RC4 y RC3
 - El PIC contiene el módulo I2C integrado

- **Puerto Serie RS232: Puerto C <7:6>**
 - **RC6 es TX y RC7 es RX**
 - **Configurar TRISC (0xF94) para RC6 salida (0) y RC7 entrada (1)**
 - **TXSTA: Transmit Status and control register (0xFAC)**
 - Bit 7 En modo asíncrono no importa
 - Bit 6 = 0: 8 TX data bits
 - Bit 5: Activar TX (1 activado / 0 desactivado)
 - Bit 4 = 0: Seleccionamos el modo asíncrono.
 - Bit 3: No implementado
 - Bit 2: **BRGH** High speed selected (Baud Rate)
 - Bit 1: Estado del registro de transmisión. 0 Vacío / 1 Lleno
 - Bit 0: Bit de paridad o 9 bit
 - **RCSTA: Receive Status and control register (0xFAB)**
 - Bit 7: Activar Puerto Serie (1 activado / 0 desactivado)
 - Bit 6 = 0: 8 RX data bits
 - Bit 5 En modo asíncrono no importa
 - Bit 4: Activar RX (1 activado / 0 desactivado)
 - Bit 3: Sólo en modo 9 bits
 - Bit 2: Indica error de trama
 - Bit 1: Indica error de desbordamiento
 - Bit 0: Bit de paridad o 9 bit

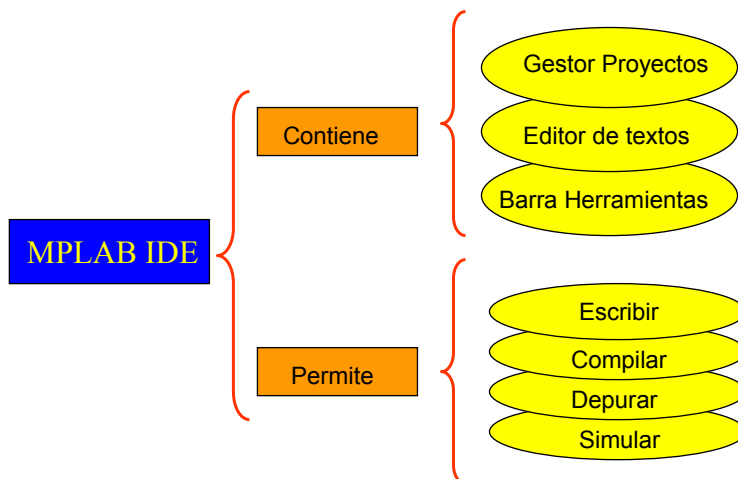
- Puerto Serie RS232: continuación...
 - **TXREG: USART Transmit Register (0xFAD)**
 - **SPBRG (0xFAF) & TXSTA(BRGH): Baud Rate Generator Register**
 - **Modo asíncrono**
 - **9600 baudios**
 - **Fosc = 4 Mhz**
- $$\begin{array}{l}
 \text{BRGH} = 0 \rightarrow \text{Baud Rate} = \text{Fosc} / (64 * (\text{SPBRG} + 1)) \\
 \text{BRGH} = 1 \rightarrow \text{Baud Rate} = \text{Fosc} / (16 * (\text{SPBRG} + 1))
 \end{array}
 \left. \vphantom{\begin{array}{l} \text{BRGH} = 0 \\ \text{BRGH} = 1 \end{array}} \right\} \text{Asíncrono}$$
- En el ejemplo anterior: SPBRG = **25 (high)** y 6 (low)
- **Interrupción de recepción: RCIF, RCIE, RCIP**
 - **Interrupción de transmisión: TXIF, TXIE, TXIP**
- **Al recibir se activa RCIF y se lee el dato en RCREG (0xFAE)**

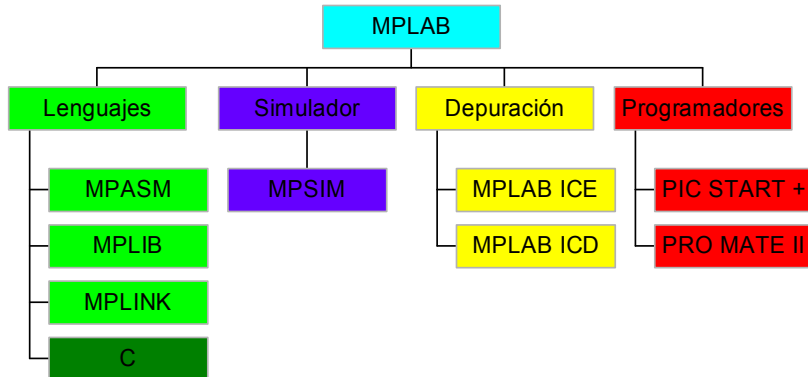
Curso Programación en C de microcontroladores PIC

Entorno de desarrollo:

MPLAB – IDE
MPLAB – ICD2
MPLAB – C18

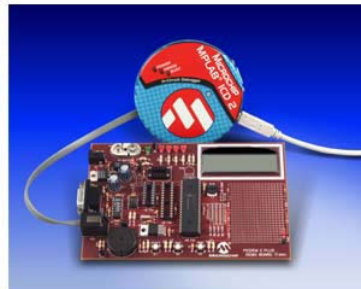
MPLAB IDE : Descripción



**MPLAB ICD2 (In Circuit Debugger)**

- Depurar el código en nuestra aplicación, utilizando el propio MCU (In-Circuit-Debugger)
- Depurar el hardware en tiempo real.
- Programar el MCU directamente utilizando el protocolo In-Circuit-Serial-Programming (ICSP)

- Válido para la familia **PIC16F87X y sucesores**
- Velocidad de emulación hasta 20Mhz
- Ejecución paso a paso / múltiples breakpoints
- Voltaje mínimo de emulación 3.0 v
- Conexión por el puerto serie y USB



1. Instalamos el MPLAB: **MPLAB-IDE**
2. Instalamos el compilador de C: **MPLAB-C1**
3. Configuramos el entorno:
 - Seleccionamos el tipo de **micro**
 - Configuramos los **bits de control**
 - (nosotros) Todo desactivado
 - (nosotros) Oscilador XT
 - Seleccionamos el tipo de **programador**
 - Configuramos el programador
 - Establecemos límite de memoria 0xFFFF
4. Localizamos las **herramientas** de trabajo
 - MCHIP_T\mpasmwin.exe -> ensamblador
 - Mcc18\c18demo.exe -> compilador de C
 - MCHIP_T\mplink.exe -> linkador

5. Creamos un **proyecto** nuevo:
 1. Proyecto nuevo: Nombre y dirección
 2. Seleccionamos herramienta trabajo: MPLAB-C18
 3. Configuramos las opciones de compilación:
 - General -> ponemos los PATH de .h y .lib
 - MPLAB-C18 -> flags de compilación.
 4. Añadimos el **script de linkado**
 - Botón derecho en *linker script* en el panel del proyecto.
 - Añadimos el fichero \mcc18\lnk\'dispositivo'.
 - Para usar el ICD2 poner fichero terminado en 'i'.
 5. Añadimos fichero **fuentes**
 - Botón derecho en *Source File* en el panel del proyecto.
 - Añadimos el fichero fuente.

Curso Programación en C

de microcontroladores PIC

Laboratorio I

Práctica I: Punteros

Enunciado:

Hacer un programa que encienda el LED RB0

- a) Utilizar declaración de variables y punteros de C
- b) Simplificar las declaraciones al máximo
- c) Analizar dichas opciones con el simulador

Notas:

No utilizar llamadas a librerías

Enunciado:

Hacer un programa que apague el LED RB0 cuando se pulsa RA4

- a) usar declaración directa de variables
- b) Comprobar únicamente el bit RA4.
- b) Programarlo con `#include <p18f452.h>`

Notas:

No utilizar llamadas a librerías

Práctica III: Sonido y efectos luminosos

Enunciado:

Completar/Corregir el programa siguiente para que cada vez que se pulse RA4:

- a) se emita un sonido por el zumbador.
- b) se incremente una variable y se muestre por los LEDS.

`valor = ( valor + 1 ) % 16;    16 = 2 ^ 4 bits.`

Práctica III: Sonido y efectos luminosos



```
1. // Programa base para la práctica 3
2.
3. #include <p18f452.h>
4.
5. void main(void) {
6.     // configuramos los puertos
7.     TRISB = 0;    // puerto B configurado como salida
8.     TRISA = 0xFF; // puerto A configurado como entrada
9.     TRISC = 0;    // puerto C configurado como salida
10.
11.     // valor inicial puertos
12.     PORTA=0;
13.     PORTB=0;
14.
15.     // bucle principal
16.
17.     if ( PORTA != 0x10 ) {
18.         PORTB = ( PORTB + 1 ) % 16;
19.         PORTC = 0xff;
20.     }
21. }
```



Práctica IV: Cadenas



Enunciado:

Utilizando el Debugger verificar cuál de las dos funciones de manejo de cadenas es la correcta.

1. Seleccionar como herramienta de debugger el MPLAB-SIM.
2. Comparar las zonas de memoria de datos y de programa mediante las opciones del menú view:
 - Programm memory
 - File Register

```
1. // Ejemplo de tratamiento de cadenas en RAM y ROM
2.
3. #include <string.h>
4.
5. // almaceno mi cadena en ROM
6. #pragma romdata rom_variables=0x200
7. const rom unsigned char micadena[]="hola mundo";
8. #pragma romdata
9.
10. #pragma udata ram_variables=0x200
11. char strbuffer[21];
12. #pragma udata
13.
14. void main(void) {
15.     // sólo una es válida
16.     strcpypgm2ram(strbuffer, micadena);
17.     strcpy(strbuffer, micadena);
18.     for (;;) ;
19. }
```

Enunciado:

- **Declarar una estructura tipo puerto que pueda ser accedida bit a bit, con nibble o directamente con un valor de byte.**
- **Usar la estructura anterior para encender el LED conectado a RB0.**

Enunciado:

Hacer un programa que muestre los 4 bits más significativos de la conversión AD en los LEDS.

1. Completar el esqueleto propuesto.
2. Considerar que la conversión es de 10 bits.
 - Colocación del resultado
 - Desplazamiento de bits adecuado.

Utilizar:

1. programación directa
2. Programación con librerías (ADC)

```
#include <p18f452.h>

void main(void) {

    // Configuración Puerto B como salida
    TRISB = ???;

    // configuramos el conversor
    ...

    // bucle principal
    while (1) {
        // esperamos a que termine la conversión en curso
        while ( ? != 0 );
        // activamos la siguiente conversión
        ...
        // activamos los LEDS
        PORTB=...;
    }
}
```

Práctica VII: Puerto Serie

Enunciado:

Hacer un servidor de ECO

1. Configurar el puerto serie para 9600 baudios y 8 bits.
2. Esperar hasta recibir un carácter por el puerto serie.
3. Enviar el carácter leído por el puerto serie.

Utilizar:

1. programación directa
2. Probar que ocurre si nos equivocamos en los baudios
3. Programación con librerías (USART)

Práctica VIII: Timer

Enunciado:

Hacer que el LED cambie de estado cada segundo.

1. Utilizar el TIMER 1 para medir el segundo.
2. Aprovechar el cristal de 32K que lleva asociado dicho timer.

Curso Programación en C microcontroladores PIC

Programación avanzada

Directivas Pragma: Section (I)

1. **#pragma sectiontype**

- Sirve para cambiar la sección actual en la que el compilador guardará la información.
- Las secciones pueden contener tanto datos como código y se pueden encontrar en memoria de datos y de programa.
- Dos tipos de secciones para cada tipo de memoria:
 - code**: instrucciones en memoria de programa
 - romdata**: datos en memoria de programa
 - udata**: variables de usuario estáticas sin inicializar
 - idata**: variables de usuario estáticas inicializadas
- Se pueden añadir secciones completando el 'Linker Script'
- Podrán ser:
 - absolute**: Indicamos el lugar en memoria de la sección
 - assigned**: El lugar de memoria se indica en el 'linker script'

- Los atributos son dos:

Overlay: Permite que varias secciones se puedan grabar en una misma zona de memoria. Las secciones involucradas deben tener el mismo nombre y pertenecer a ficheros fuente distintos. Si son de tipo access o absolutas lo serán por igual todas.

Access: La sección se sitúa en una zona de memoria directamente accesible por las instrucciones. Las variables llevarán asociado el atributo NEAR. La región de acceso se define por ACCESSBANK en el linker-script.

- linker script

Es un fichero que proporciona diversa información al 'linker':

- Define las posiciones en memoria de las secciones.
- Define el tamaño y situación de la pila (stack) software.
- Indica la situación de las palabras de configuración.

Ejemplo práctico:

Abrimos el proyecto "variables" y analizamos la localización de variables y el cambio de código al compilar con -Oa- (near off) y -Oa+ (near on).

Ejemplo práctico notas:

1. La opción **-Oa-** desactiva la opción de que las variables se sitúen en la zona de acceso. Es decir estarán a partir de la posición 0x080, y para acceder a ellas se utilizará el 'bank switching'. Opción NEAR OFF
2. La opción **-Oa+** insta al compilador para situar las variables en el banco de acceso. Es decir estarán en la zona RAM comprendida hasta 0x080. Para acceder a ellas se utiliza la opción 'a=0' de las instrucciones. Hay que tener en cuenta, que al declarar cualquier variable que hayamos situado fuera del banco de acceso, hay que usar **far**. Opción NEAR ON.
3. Para generar el mapa de variables es necesario utilizar la opción "generate map file". En el menú de 'opciones de compilación -> linker' dentro de la sección proyectos.
4. Para ver el código generado en ensamblador se puede utilizar la opción 'Disassembly' del menú 'ver'.

Directivas Pragma: varlocate

2. #pragma varlocate 'banco' 'nombre-variable'

#pragma varlocate 'mi_sección' 'nombre-variable'

- Esta directiva le dice al compilador, en tiempo de *linkado*, donde se puede encontrar la variable especificada. De tal forma que el compilador puede optimizar el cambio de banco.
- Nosotros debemos especificar en los ficheros fuente que variables están afectadas.

Fichero 1

```
#pragma udata bank1=0x100
signed char c1;
signed char c2;
```

Fichero 2

```
#pragma varlocate 1 c1
extern signed char c1;

#pragma varlocate 1 c2
extern signed char c2;

Vvoid main(void) {
    c1 += 5;
    c2 += 5; // NO
    MOVLB
}
```

Directivas Pragma: Interrupt (I)

3. Interrupciones

1. El compilador transfiere el flujo del programa a la función ISR para procesar el evento que la ha producido.
2. Antes de transferir el control guarda el contexto. Lo mínimo es: WREG, BSR, STATUS.
3. Utilizan una **zona temporal** para guardar las variables y datos. En concreto una zona tipo 'access', 'udata' y nombre 'fname_tmp'.
4. Las funciones se declaran **sin parámetros** y **no devuelven** nada. La razón es que se producen de forma asíncrona con el flujo del programa.
5. Las variables globales accesibles por estas funciones se deben declarar con el modificador **volatile**.
6. No se deben llamar desde otras funciones. La razón es que devuelven el control con RETFIE en lugar de con RETURN.
7. El tiempo que tarde en atenderse la rutina depende del procesador en enterarse, del tiempo que tarda en leer el vector y del tarda el C18 en guardar el contexto.

#pragma interruptlow 'nombre-función'

Indica al compilador que 'nombre-función' es la rutina de servicio de atención a la interrupción con prioridad baja.

- Utiliza la pila software para guardar el contexto.
- Sale de la interrupción con el retorno normal.
- Sobrecarga de 10 acceso a WORD respecto a I. con alta prioridad.
- Se pueden encadenar. Indicarlo por software al procesador.

#pragma interrupt 'nombre-función'

Indica al compilador que 'nombre-función' es la rutina de servicio de atención a la interrupción con prioridad alta.

- Utiliza los registros SHADOW para guardar el contexto.
- Sale de la interrupción con un retorno rápido.
- Se usan para secciones críticas en tiempo.

Vectores de interrupción

- Nosotros tenemos que indicarle al compilador como hacer que el vector de interrupción apunte a la rutina de servicio.
- Lo que se hace es colocar la instrucción GOTO en la dirección del vector de interrupción para transferir el control a la ISR.

Vector Int. RESET	0x0000	} User memory
...		
Vector Int. High	0x0008	
...		
Vector Low. High	0x0018	
On chip memory	0x0019 – 0x7FFF	
0	0x8000 – 0x1FFFFFF	

```

1. #include <p18f452.h>
2. void low_isr(void);
3.
4. // configuro el vector de interrupción LOW
5. #pragma code low_vector=0x18
6. void low_interrupt(void) {
7.     _asm GOTO low_isr _endasm
8. }
9.
10. // Esta es la Rutina de Servicio de la Interrupción
11. #pragma interruptlow low_isr
12. void low_isr (void) {
13.     // instrucciones
14. }

```

Opciones para guardar correctamente el CONTEXTTO

- **#pragma interrupt high_ISR save= *mi-var1* , *mi-var2***
- **#pragma interrupt high_ISR save=section("*mi-seccion*")**
- Si la ISR llama a otra función:
#pragma interrupt high_ISR save=section("*tmpdata*")
- Si la ISR llama a una función que devuelve 16 bits:
#pragma interrupt high_ISR save=section("*tmpdata*") , PROD
- Si la ISR utiliza la librería matemática o llama a una función que devuelve 24 o más bits:
#pragma interrupt high_ISR save=section("*MATH_DATA*")
- Las opciones las podemos encadenar separándolas por comas:
#pragma interrupt high_ISR save=section("*tmpdata*") , PROD, *mi-var1*

Opciones para guardar correctamente el CONTEXTO

- `#pragma interrupt high_ISR save= mi-var1 , mi-var2`
- `#pragma interrupt high_ISR save=section("mi-seccion")`
- Si la ISR llama a otra función:
`#pragma interrupt high_ISR save=section(".tmpdata")`
- Si la ISR llama a una función que devuelve 16 bits:
`#pragma interrupt high_ISR save=section(".tmpdata") , PROD`
- Si la ISR utiliza la librería matemática o llama a una función que devuelve 24 o más bits:
`#pragma interrupt high_ISR save=section("MATH_DATA")`
- Las opciones las podemos encadenar separándolas por comas:
`#pragma interrupt high_ISR save=section(".tmpdata") , PROD, mi-var1`

Ejercicio práctico:

Hacer un programa que apague el LED RB0 cuando se pulse RA4. Además, cada segundo tendrá que cambiar el estado del LED RB1

Método:

1. Utilizar el TIMER 1 para producir una interrupción cada segundo.
2. Hacer que la rutina de interrupción cambie el estado del LED.
3. El programa principal se tendrá que encargar del pulsador.

En realidad es una directiva 'Section' que actúa sobre la memoria de configuración de los micros PIC. Esta zona se define en el 'linker-script' asociado a cada micro y se llama CONFIG

```
CODEPAGE NAME=config START=0x300000 END=0x300007 PROTECTED
SECTION NAME=CONFIG ROM=config
```

```
1. #pragma romdata CONFIG
2. _CONFIG_DECL ( _CP_ON_1L,
3.               _OSCS_ON_1H & _OSC_XP_1H,
4.               _PWRT_ON_2L & _BOR_OFF_2L & _BORV_42_2L,
5.               _WDT_OFF_2H & _WDTPS_1_2H,
6.               _CCP2MUX_OFF_3H,
7.               _CONFIG4L_DEFAULT);
8. #pragma romdata
```

- MPLAB C-18 integra un ensamblador interno con una sintaxis como la del MPASM
- Para introducir un bloque en ensamblador hay que empezar con **_asm** y terminar con **_endasm**.
- Las diferencias con el MPASM:
 - No admite directivas.
 - Los comentarios son estilo C.
 - El Radix por defecto es decimal.
 - Las etiquetas incluyen dos puntos.
 - Los datos se especifican con notación de C. (0x12F).
 - Las instrucciones de Acceso a tablas tienen que tener los mnemónicos enteros.
 - Todos los operandos de las instrucciones se deben especificar.

Como llamar a funciones de C desde Ensamblador

1. Las funciones de C se comportan como globales salvo que se declaren *static*.
2. La función de C se declara *EXTERN* en el código en ensamblador.
3. Para llamar a la función se utiliza *CALL* o *RCALL*.
4. Los parámetros se llevan a la pila. De derecha a izquierda y el byte menos significativo primero.
5. Los parámetros estáticos son globales. Se accede con **_nombrefuncion:pos**

char add (auto char x, auto char y); // función de C

```

•          EXTERN add ;
•          ...
•          MOVLW 0x65
•          MOVWF POSTINC1 ; y=0x65 lo dejamos en la pila
•          MOVLW 0x61
•          MOVWF POSTINC1 ; x=0x61 lo dejamos en la pila
•          CALL  add      ; add (0x61,0x65)
•          MOVWF resultado ; el resultado te lo devuelven en WREG. Si el valor devuelto es de 16 bits, estará en
•                          ; PROD
    
```

Como llamar a funciones Ensamblador desde C

- Las funciones en ensamblador hay que declararlas globales.
- La función se declara *EXTERN* en el código en C.
- Debe devolver los valores siguiendo la norma del C18.
- La función se llama como si se tratase de una en C.

```

1.          UDATA_ACS
2.  delay_temp RES 1
3.          CODE
4.  asm_delay ...
5.          RETURN
6.          GLOBAL asm_delay ; la exportamos para que la vea el 'linker'
7.          END
    
```

```

1.  extern void asm_delay( void );
2.  void main(void) {
3.      asm_delay();
4.  }
    
```

} Código en C desde donde llamo a la función

Como llamar variables de C desde ensamblador

- Las variables se declaran globales en C.
- Las variables se declaran extern en ensamblador
- Se utilizan directamente.

Como llamar variables de ASM desde C

- Las variables se declaran globales en ensamblador.
- Las variables se declaran extern en el C.
- Se utilizan directamente.

1. Declaración de los registros de un procesador

- Es un fichero que contiene las declaraciones externas de los recursos de un procesador.
- Son ficheros de cabecera, es decir *'pxxx.h'*
- Se localizan en */mcc18/h/p18f452.h*
- Define ciertas funciones declaradas como macros.
Nop(), Sleep(), CkrWdt(), Reset() y operaciones con bits.

2. Definición de los registros de un procesador

- Es un fichero en ensamblador que contiene la definición de los registros.
- Se compila y se genera un *fichero.o* que se linkara con nuestra aplicación.
- Se localizan en */mcc18/src/proc/p18f452.asm*

3. Linker Script

- Contiene información sobre uso de la memoria: pila, config, y zonas de memoria.
- Se encuentra en *mcc18/lkr/18f452.lkr*
- Para ejecutar con MPLAB-ICD2 usar los archivos terminados terminados en *'i'*.

1. Declaración de los registros de un procesador

- Es un fichero que contiene las declaraciones externas de los recursos de un procesador.
- Son ficheros de cabecera, es decir *'pxxxx.h'*
- Se localizan en */mcc18/h/p18f452.h*
- Define ciertas funciones declaradas como macros.
Nop(), Sleep(), ClrWdt(), Reset() y operaciones con bits.

2. Definición de los registros de un procesador

- Es un fichero en ensamblador que contiene la definición de los registros.
- Se compila y se genera un *fichero.o* que se linkara con nuestra aplicación.
- Se localizan en */mcc18/src/proc/p18f452.asm*

3. Linker Script

- Contiene información sobre uso de la memoria: pila, config, y zonas de memoria.
- Se encuentra en *mcc18/lkr/18f452.lkr*
- Para ejecutar con MPLAB-ICD2 usar los archivos terminados terminados en 'i'.

1. /mcc18/lib

- Directorio con las librerías y códigos de inicio ya compiladas.

2. /mcc18/src/startup

- Contiene el código de inicio del C-18.
- El vector de reset apuntará a este código que después dará paso a nuestra aplicación en un bucle infinito. (si queremos poner código antes habrá que ponerlo al principio de *_entry()*)
 - *c018.o* inicializa la pila y salta a main
 - *c018i.o* inicializa la pila, las variables inicializadas y salta a main. (default)
 - *Cc018iz.o* inicializa la pila, las variables inicializadas y no inicializadas y salta a main.

3. Librerías independientes del procesador (/mcc18/src/...)

- Las fuentes se encuentran en estos directorios: Math, delays, ctype, string, stdlib
- Forman la *clib.lib*

4. Librerías dependientes del procesador (/mcc18/src/...)

- Las fuentes se encuentran en estos directorios: pmc , proc
- Forman la *p18f452.lib*
- Contienen las funciones de los periféricos hardware y software.

Recursos ocupados	Uso principal	Guardados automáticamente
PC	Control de la ejecución	X
WREG	Cálculos intermedios	X
STATUS	Resultado de cálculos	X
BSR	Selección de bancos	X
PROD	Valores de retorno y cálculos	
Sección .tmpdata	Cálculos intermedios	
FSR0	Puntero a RAM	X
FSR1	Puntero a Pila	X
FSR2	Puntero a marco	X
TBLPTR	Acceso a variables situadas en la memoria de programa. Tablas	
TABLAT		
PCLATH	Puntero para llamadas a funciones	
PCLATU		
Sección MATH_DATA	Utilizada por la librería matemática	

Curso Programación en C de microcontroladores PIC

Laboratorio II

Práctica: Servidor

Enunciado:

Hacer un programa servidor residente en el PIC que reciba comandos por el puerto serie y actúe sobre cada uno de los periféricos de la PICDEM2.

Se seguirán los siguientes pasos:

1. Programar el puerto serie para poder recibir una trama de 4 bytes. Cuando estemos esperando a recibir el primer carácter de trama encenderemos RB0. Una vez recibida la trama proceder a su análisis.
2. Utilizar el primer byte como identificador de comando.
 - 'a' -> activar el zumbador.
 - 'b' -> enviar el estado del pulsador RA4 al PC. Enviar la cadena ON y OFF en ASCII.
 - 'c' -> El segundo byte es un valor entre 0 y 7 en ASCII. Representarlo en los LEDS RB1:RB3.

Práctica: Servidor

- 'd' -> los tres siguientes bytes, indican un valor de un registro de la memoria de datos. Leer ese valor y enviarlo al PC en hexadecimal.
- 'e' -> apagar el zumbador

Notas

Se puede introducir un carácter de trama para obligar a ir al principio de la recepción. Forma de sincronismo.

Se deberían de emplear las interrupciones para mantener el zumbador sonando mientras se siguen procesando comandos.

También se podrían utilizar las interrupciones para recibir por el puerto serie. De esa forma no tenemos que estar pendientes de la recepción de trama.

Documentación

- www.microchip.com
- CDROM
 - Getting started with C
 - C18 user guide
 - Librerías C18
 - Picdem2 user guide
 - Datasheet 18f452